Number System Basics

1. What do you mean by number system?

A number system is a way to represent numbers using a set of symbols or digits. It defines how numbers are expressed and interpreted. The most common number system is the decimal system (base 10), but there are others like binary (base 2), octal (base 8), and hexadecimal (base 16).

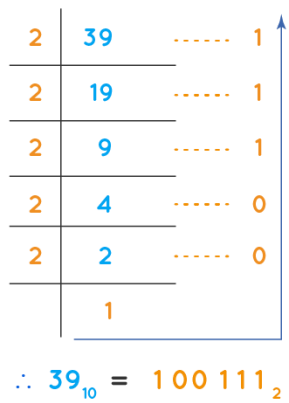
2. What are the types of number systems? Describe briefly.

- Decimal (Base 10): Uses digits 0 to 9. It is the standard number system used in daily life.
- Binary (Base 2): Uses digits 0 and 1. Used in computers and digital systems.
- Octal (Base 8): Uses digits 0 to 7. Often used in computing as a shorthand for binary.
- Hexadecimal (Base 16): Uses digits 0 to 9 and letters A to F. Common in computer programming and digital electronics.

3. Convert $(39)_{10}$ (Decimal) to Binary, then Binary to Decimal step by step:

Decimal to Binary:

39 ÷ 2 = 19 remainder 1
19 ÷ 2 = 9 remainder 1
9 ÷ 2 = 4 remainder 1
4 ÷ 2 = 2 remainder 0
2 ÷ 2 = 1 remainder 0
1 ÷ 2 = 0 remainder 1
=> $(39)_{10} = (100111)_2$



Binary to Decimal:

$(100111)_2 = 1\times2^5 + 0\times2^4 + 0\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 = 32 + 0 + 0 + 4 + 2 + 1 = 39$

4. Convert $(39)_8$ (Octal) to Decimal and Binary step by step:

Octal to Decimal:

$(39)_8 = 3\times8^1 + 9\times8^0 = 24 + 9 = (33)_{10}$

Decimal to Binary:

33 ÷ 2 = 16 remainder 1
16 ÷ 2 = 8 remainder 0
8 ÷ 2 = 4 remainder 0
4 ÷ 2 = 2 remainder 0
2 ÷ 2 = 1 remainder 0
1 ÷ 2 = 0 remainder 1
=> $(33)_{10} = (100001)_2$

5. Convert $(39)_8$ Octal to Hexadecimal step by step:

$(39)_8 = (33)_{10} = (100001)_2$
Group binary into 4-bit: 0010 0001
=> Hexadecimal = 2 1 => $(21)_{16}$

6. Names of some codes used in digital systems:

- ASCII (American Standard Code for Information Interchange)
- Unicode
- EBCDIC (Extended Binary Coded Decimal Interchange Code)
- Gray Code
- BCD (Binary-Coded Decimal)

7. Discuss BCD Code briefly:

BCD stands for Binary-Coded Decimal. It represents each decimal digit (0-9) with its 4-bit binary equivalent. For example, the number 39 is written in BCD as:
3 = 0011, 9 = 1001 => BCD = 0011 1001

8. Applications and Challenges of Number Systems:

Applications:
- Used in computers and digital systems
- Essential for programming, networking, and electronic devices
- Binary is used for logic circuits and data storage

Challenges:
- Conversion between number systems can be complex
- Understanding different bases requires practice
- Errors in conversion may lead to system bugs

9. Convert $(1056)_{16}$ (Hexadecimal) to Octal:

$(1056)_{16}$ = 0001 0000 0101 0110 (binary)
Group into 3-bits: 001 000 000 101 011 000
=> Octal = 1 0 0 5 3 0 => $(100530)_8$

10. Name of codes (ASCII, UNICODE):

- ASCII: Represents English characters using 7 or 8 bits.
- UNICODE: Supports characters from almost all languages; uses up to 32 bits.

11. De Morgan's Laws - Statement and Simplification:

First Law: (A + B)' = A'.B'
Second Law: (A.B)' = A' + B'

Application of Number System:
- Used in digital electronics, networking, and automation.

Challenge:
- Errors during manual conversion or misinterpretation in data representation.

Detailed Explanation of De Morgan's Laws

First Law: $\overline{(A + B)} = \overline{A} \cdot \overline{B}$

Meaning:
The NOT of (A OR B) is equal to (NOT A) AND (NOT B).

Example:
If A = 1 (True), B = 0 (False)
A + B = 1 → $\overline{(A + B)}$ = 0
A' = 0, B' = 1 → $\overline{A} \cdot \overline{B}$= 0 AND 1 = 0
=> Both sides give the same result.

Second Law: $\overline{(A \cdot B)} = \overline{A} + \overline{B}$

Meaning:
The NOT of (A AND B) is equal to (NOT A) OR (NOT B).

Example:
If A = 1, B = 1
A · B = 1 → $\overline{(A \cdot B)}$ = 0
A' = 0, B' = 0 → $\overline{A} + \overline{B}$ = 0 OR 0 = 0
=> Both sides give the same result.

Why It's Useful

De Morgan's laws are useful in:
- Simplifying logic expressions
- Designing digital circuits
- Converting between NAND and NOR logic

Truth Tables Explained

A truth table shows all possible input combinations to a logic gate or expression and their corresponding output.

1. NOT Gate

A (Input)	A' (Output)
0	1
1	0

2. AND Gate

A	B	A · B (AND)
0	0	0
0	1	0
1	0	0
1	1	1

3. OR Gate

A	B	A + B (OR)
0	0	0
0	1	1
1	0	1
1	1	1

4. NAND Gate

A	B	A · B	(A · B)' (NAND)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

5. De Morgan's First Law: (A + B)' = A' · B'

A	B	A + B	(A + B)'	A'	B'	A' · B'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

5. What is Boolean Algebra?

Boolean algebra is a branch of mathematics that deals with variables that have two possible values: true (1) and false (0)'

Basic Laws

- **Identity Laws**:
  - $A + 0 = A$
  - $A \cdot 1 = A$
- **Domination Laws**:
  - $A + 1 = 1$
  - $A \cdot 0 = 0$
- **Complement Laws**:
  - $A + \overline{A} = 1$
  - $A \cdot \overline{A} = 0$

Commutative, Associative, and Distributive Laws

- **Commutative Laws**:
  - $A + B = B + A$
  - $A \cdot B = B \cdot A$
- **Associative Laws**:
  - $A + (B + C) = (A + B) + C$
  - $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
- **Distributive Laws**:
  - $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
  - $A + (B \cdot C) = (A + B) \cdot (A + C)$

Basic ADDERS

A half-adder adds two bits and produces a sum and a carry output. Adders are important in computers and also in other types of digital systems in which numerical data are processed.

The Half-Adder

Recall the basic rules for binary.

$0 + 0 = 0$

$0 + 1 = 1$

$1 + 0 = 1$

$1 + 1 = 10$

The operations are performed by a logic circuit called a half-adder.

The half-adder accepts two binary digits on its inputs and produces two binary digits on its outputs, a sum bit and a carry bit.

A half-adder is represented by the logic symbol in Fig.(7-1).

Half-Adder Logic: From the operation of the half-adder as stated in Table 7-1, expressions can be derived for the sum and the output carry as functions of the inputs. Notice that the output Carry (C_{out}) is a 1 only when both A and B are 1s: therefore. C_{out} can be expressed

$$C_{out} = AB$$

Now observe that the sum output ($\sum$) is a 1 only if the input variables A and B, are not equal. The sum can therefore be expressed as the exclusive-OR of the input variables.

$$\sum = A \oplus B$$

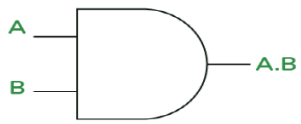
Logic Gates

- A logic gates are an electronic circuit that are designed by using electrical components like diodes, transistors, resistors, and more.
- It is used to perform logical operations based on the inputs provided to it and gives logical output that can be either high(1) or low(0).
- The operation of logic gates is based on the Boolean algebra or mathematics.
- Logic gate founds its uses in our day to day basis such as in the architecture of our telephone, laptops, tablets an memory devices.

AND Gate

An AND gate is used to perform logical Multiplication of binary input. The Output state of the AND gate will be high(1) if both the input are high(1) ,else the output state will be low(0) if any of the input is low(0).

2- Input AND Gate

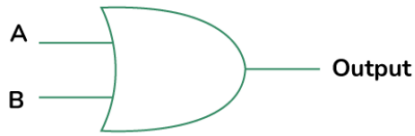


Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE is most widely used digital logic circuit. The output state of OR gate will be high i.e.,(1) if any of the input state is high or 1, else output state will be low i.e., 0.

2-Input OR Gate

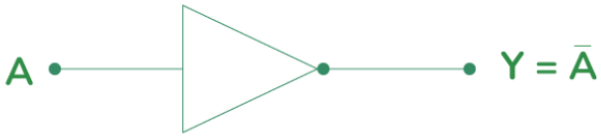


Truth Table

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

In digital electronics, the NOT gate is one of the basic logic gate having only a single input and a single output. It is also known as inverter or inverting buffer. When the input signal is “low” the output signal is “high” and vice-versa.

NOT Gate



Truth Table

A (Input)	Y = Ā (Output)
0	1
1	0

By- md.pranto